

ASSIGNMENT-13.3

2303A51242

BATCH-05

Task 1: Refactoring – Removing Code Duplication

PROMPT: Refactor the following legacy Python code to remove duplicated logic.

Requirements:

1. Create reusable functions to calculate the area and perimeter of a rectangle.
2. Eliminate all repeated code blocks.
3. Use proper function definitions and parameters.
4. Add clear docstrings explaining what each function does, including parameters and return values.
5. Keep the program simple and readable.
6. Print the area and perimeter for the rectangles (5,10), (7,12), and (10,15) using the reusable functions.

Legacy Code:

```
print("Area of Rectangle:", 5 * 10)
print("Perimeter of Rectangle:", 2 * (5 + 10))
print("Area of Rectangle:", 7 * 12)
print("Perimeter of Rectangle:", 2 * (7 + 12))
print("Area of Rectangle:", 10 * 15)
print("Perimeter of Rectangle:", 2 * (10 + 15))
```

Return the improved Python code with proper structure and documentation.

CODE:

```
def calculate_area(length, width):
    """
    Calculate the area of a rectangle.

    Parameters:
    length (float): The length of the rectangle.
    width (float): The width of the rectangle.

    Returns:
    float: The area of the rectangle.
    """
    return length * width

def calculate_perimeter(length, width):
    """
    Calculate the perimeter of a rectangle.

    Parameters:
    length (float): The length of the rectangle.
    width (float): The width of the rectangle.

    Returns:
    float: The perimeter of the rectangle.
    """
    return 2 * (length + width)

# Test the functions with the given rectangles
rectangles = [(5, 10), (7, 12), (10, 15)]

for length, width in rectangles:
    print(f"Area of Rectangle ({length}, {width}): {calculate_area(length, width)}")
    print(f"Perimeter of Rectangle ({length}, {width}): {calculate_perimeter(length, width)}")
```

OUTPUT:

```
Drive/Desktop/AI ASSISTANT CODING/assignment-13.3.py"
Area of Rectangle (5, 10): 50
Perimeter of Rectangle (5, 10): 30
Area of Rectangle (5, 10): 50
Perimeter of Rectangle (5, 10): 30
Area of Rectangle (7, 12): 84
Perimeter of Rectangle (7, 12): 38
Perimeter of Rectangle (7, 12): 38
Area of Rectangle (10, 15): 150
Area of Rectangle (10, 15): 150
Perimeter of Rectangle (10, 15): 50
PS C:\Users\kamu1\OneDrive\Desktop\AI ASSISTANT CODING>
```

Explanation: This code improves the legacy program by **removing repeated calculations** and using **reusable functions**.

The `calculate_area()` function computes the **area of a rectangle**, and the `calculate_perimeter()` function computes the **perimeter**. A list of rectangle dimensions is stored, and a **loop calls these functions** to print the area and perimeter for each rectangle. This makes the code **cleaner, reusable, and easier to maintain**.

Task 2: Refactoring – Optimizing Loops and Conditionals

PROMPT: Refactor the following Python code to optimize inefficient nested loops.

Requirements:

1. Replace the nested loops with a more efficient approach using sets or list comprehensions.
2. Improve performance for searching elements in the list.
3. Keep the code simple and readable.
4. Print whether each name in `search_names` exists in `names`.
5. Briefly compare the performance of the original nested-loop approach and the optimized solution.

Legacy Code:

```
names = ["Alice", "Bob", "Charlie", "David"]
```

```
search_names = ["Charlie", "Eve", "Bob"]
```

```
for s in search_names:
```

```
    found = False
```

```
    for n in names:
```

```
        if s == n:
```

```
            found = True
```

```
    if found:
```

```
        print(f'{s} is in the list')
```

```
    else:
```

```
        print(f'{s} is not in the list')
```

Return the optimized Python code and a short explanation of the performance improvement.

Code:

```
Return the optimized Python code and a short explanation of the performance improvement."""  
# Optimized Code  
names = ["Alice", "Bob", "Charlie", "David"]  
search_names = ["Charlie", "Eve", "Bob"]  
# Convert names list to a set for O(1) average time complexity lookups  
names_set = set(names)  
  
for s in search_names:  
    if s in names_set:  
        print(f"{s} is in the list")  
    else:  
        print(f"{s} is not in the list")  
# Performance Improvement Explanation:  
# The original code uses nested loops, resulting in O(n*m) time complexity, where n is the length of names and m is the length of
```

Output:

```
PROBLEMS 5 OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE  
ASSISTANT CODING/assignment-13.3.py"  
Charlie is in the list  
Eve is not in the list  
Bob is in the list  
PS C:\Users\kamul\OneDrive\Desktop\AI ASSISTANT CODING>
```

Explanation: The original code uses nested loops, resulting in $O(n*m)$ time complexity, where n is the length of names and m is the length of search_names. The optimized code converts the names list into a set, which allows for $O(1)$ average time complexity for lookups. This reduces the overall time complexity to $O(n + m)$, where n is the time to create the set and m is the time to check each search name against the set. This results in significantly improved performance, especially as the size of the names list grows.

Task 3: Refactoring – Extracting Reusable Functions

PROMPT: Refactor the following legacy Python code by extracting reusable functions.

Requirements:

1. Create a function called `calculate_total(price)`.
2. The function should calculate tax at 18% and return the total price (price + tax).
3. Remove duplicated code and make the program modular and reusable.
4. Add a proper docstring describing the function, its parameter, and return value.

5. Use the function to calculate and print the total price for 250 and 500.

Legacy Code:

```
price = 250
tax = price * 0.18
total = price + tax
print("Total Price:", total)

price = 500
tax = price * 0.18
total = price + tax
print("Total Price:", total)
```

Return the improved Python code with clean structure and documentation.

Code:

```
Return the improved Python code with clean structure and documentation."""
def calculate_total(price):
    """
    Calculate the total price including tax.

    Parameters:
    price (float): The original price of the item.

    Returns:
    float: The total price after adding 18% tax.
    """
    tax = price * 0.18
    total = price + tax
    return total

# Calculate and print total price for 250
price1 = 250
total1 = calculate_total(price1)
print("Total Price for 250:", total1)

# Calculate and print total price for 500
price2 = 500
total2 = calculate_total(price2)
print("Total Price for 500:", total2)
```

Output:

```
PROBLEMS 5 OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN
PS C:\Users\kamul\OneDrive\Desktop\AI ASSISTANT CODING> & C:/U
ASSISTANT CODING/assignment-13.3.py"
Total Price for 250: 295.0
ASSISTANT CODING/assignment-13.3.py"
Total Price for 250: 295.0
Total Price for 500: 590.0
PS C:\Users\kamul\OneDrive\Desktop\AI ASSISTANT CODING>
```

Explanation: The refactored code defines a function called `calculate_total` that takes a price as an argument and calculates the total price by adding an 18% tax. The function includes a docstring that describes its purpose, parameters, and return value. The code then uses this function to calculate and print the total price for both 250 and 500, eliminating the duplicated code and making the program more modular and reusable.

Task 4: Refactoring – Replacing Hardcoded Values with Constants

PROMPT: Refactor the following Python code to replace hardcoded values (magic numbers) with named constants.

Requirements:

1. Identify the hardcoded values in the code.
2. Create constants such as `PI` and `RADIUS`.
3. Use these constants to calculate the area and circumference of the circle.
4. Make the code cleaner, more readable, and maintainable.
5. Add a short comment explaining the constants.

Legacy Code:

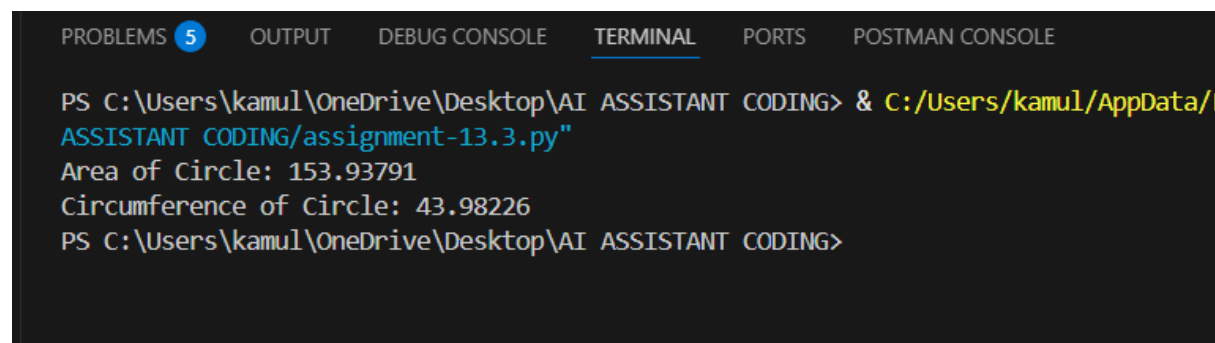
```
print("Area of Circle:", 3.14159 * (7 ** 2))
print("Circumference of Circle:", 2 * 3.14159 * 7)
```

Return the improved Python code using constants.

CODE:

```
Return the improved Python code using constants."""
# Constants for the circle calculations
PI = 3.14159 # The mathematical constant Pi
RADIUS = 7 # The radius of the circle
# Calculate the area of the circle using the formula A = π * r^2
area_of_circle = PI * (RADIUS ** 2)
# Calculate the circumference of the circle using the formula C = 2 * π * r
circumference_of_circle = 2 * PI * RADIUS
# Print the results
print("Area of Circle:", area_of_circle)
print("Circumference of Circle:", circumference_of_circle)
```

OUTPUT:

A screenshot of a code editor's terminal window. The terminal has tabs for 'PROBLEMS' (with a blue circle containing the number 5), 'OUTPUT', 'DEBUG CONSOLE', 'TERMINAL' (which is selected and underlined), 'PORTS', and 'POSTMAN CONSOLE'. The terminal text shows a command prompt 'PS C:\Users\kamul\OneDrive\Desktop\AI ASSISTANT CODING>' followed by a command to run a Python script: '& C:/Users/kamul/AppData/ASSISTANT CODING/assignment-13.3.py'. The output of the script is displayed: 'Area of Circle: 153.93791' and 'Circumference of Circle: 43.98226'. The prompt then returns to 'PS C:\Users\kamul\OneDrive\Desktop\AI ASSISTANT CODING>'.

```
PROBLEMS 5 OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE

PS C:\Users\kamul\OneDrive\Desktop\AI ASSISTANT CODING> & C:/Users/kamul/AppData/
ASSISTANT CODING/assignment-13.3.py"
Area of Circle: 153.93791
Circumference of Circle: 43.98226
PS C:\Users\kamul\OneDrive\Desktop\AI ASSISTANT CODING>
```

Explanation: By defining PI and RADIUS as constants, we have made the code more readable and maintainable. If we need to change the radius or use a more precise value of Pi in the future, we can simply update the constants without having to search through the code for hardcoded values.

Task 5: Refactoring – Improving Variable Naming and Readability

PROMPT: Refactor the following Python code to improve variable naming and readability.

Requirements:

1. Replace unclear variable names (a, b, c) with descriptive names.
2. Add inline comments explaining the logic of the program.
3. Keep the program simple and readable.
4. Ensure the output remains the same as the original code.
5. Provide a short explanation of the changes made

Legacy Code:

```
a = 10
```

```
b = 20
```

```
c = a * b / 2
```

```
print(c)
```

Return the improved Python code with better variable names, comments, and explanation.

CODE:

```
Return the improved Python code with better variable names, comments, and explanation."""
# Improved Python code with better variable names and comments
base = 10 # The base of the triangle
height = 20 # The height of the triangle
area = base * height / 2 # The area of the triangle calculated using the formula A = (1/2) * base * height
print(area)
# Explanation of changes made:
# 1. Renamed variable 'a' to 'base' to clearly indicate that it represents the base of the triangle.
# 2. Renamed variable 'b' to 'height' to clearly indicate that it represents the height of the triangle.
# 3. Renamed variable 'c' to 'area' to clearly indicate that it represents the area of the triangle.
# 4. Added inline comments to explain the purpose of each variable and the calculation being performed. This improves the readability of the code.
```

OUTPUT:

```
PROBLEMS 5 OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE
PS C:\Users\kamul\OneDrive\Desktop\AI ASSISTANT CODING> & C:/Users/kamul/OneDrive/Desktop/AI ASSISTANT CODING/assignment-13.3.py
100.0
PS C:\Users\kamul\OneDrive\Desktop\AI ASSISTANT CODING>
```

Explanation:

#1. Renamed variable 'a' to 'base' to clearly indicate that it represents the base of the triangle.

2. Renamed variable 'b' to 'height' to clearly indicate that it represents the height of the triangle.

3. Renamed variable 'c' to 'area' to clearly indicate that it represents the area of the triangle.

4. Added inline comments to explain the purpose of each variable and the calculation being performed. This improves the readability and understanding of the code.